

UNIT I

What is Software Testing?

Software Testing is a method to assess the functionality of the software program. The process checks whether the actual software matches the expected requirements and ensures the software is bug-free.

The purpose of software testing is to identify the errors, faults, or missing requirements in contrast to actual requirements. It mainly aims at measuring the specification, functionality, and performance of a software program or application.

What is a bug?

An issue or problem in the software that causes unexpected behavior. Bugs are often detected during the development environment during testing.

What is Error?

A mistake made by a programmer during coding. Errors can be caused by negligence, miscommunication, inexperience, or complexity.

What is Defect?

A non-conformance to a specification or requirement. Defects can be found during the development phase while unit testing. They can be caused by giving wrong input or code errors.

What is Failure?

The inability of the software to perform its intended function. Failures can occur when the software doesn't perform as expected during execution, or when defects go undetected.

What is Fault?

A fault is a defect or error that causes a failure when executed. Understanding these terms is important to ensure reliable and high-quality software product delivery.

What is White Box Testing?

White box testing techniques analyse the internal structures

the used data structures, internal design, code structure, and the working of the software rather than just the functionality as in black box testing. It is also called glass box testing or clear box testing or structural testing.

White Box Testing is also known as transparent testing or open box testing.

What is Black Box Testing?

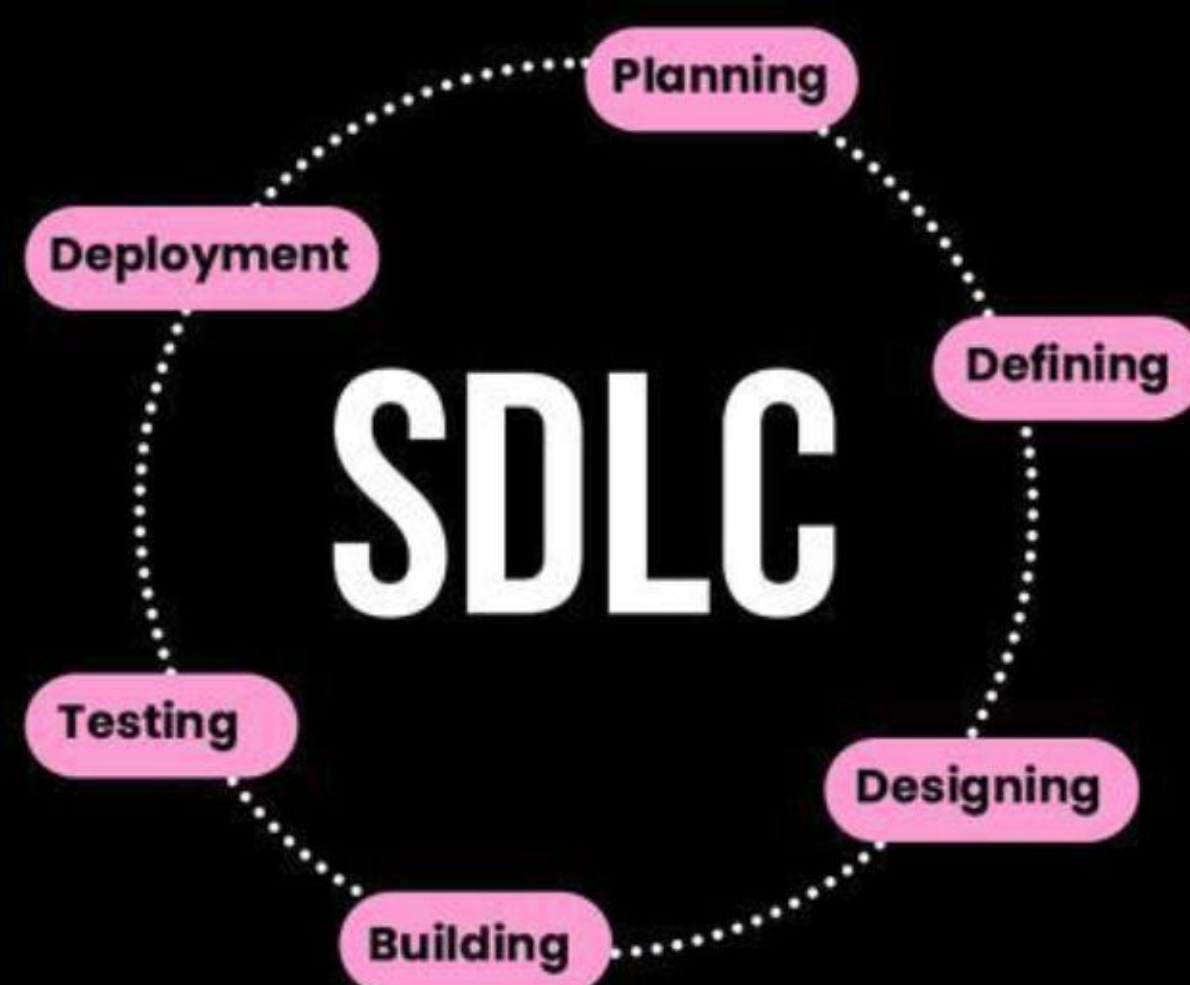
Black-box testing is a type of software testing in which the tester is not concerned with the software's internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements. The tester only focuses on the input and output of the software.

What is SDLC?

SDLC is a process followed for a software project, within a software organization. It consists of a detailed plan describing how to develop, maintain, replace and alter or enhance specific software.

SDLC is a process followed for software building within a software organization. The life cycle defines a methodology for improving the quality of software and the overall development process. Software development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software.

The following figure is a graphical representation of the various stages of a typical SDLC.



Stage 1: Planning and Requirement Analysis

Requirement analysis is the most important and fundamental

stage in SDLC. It is performed by the senior members of the team with inputs from the customer, the sales department, market surveys and domain experts in the industry.

This information is then used to plan the basic project approach and to conduct product feasibility study in the economical, operational and technical areas.

Stage 2: Defining Requirements

Once the requirement analysis is done the next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts.

This is done through an SRS (Software Requirement Specification) document which consists of all the product requirements to be designed and developed during the project life cycle.

Stage 3: Designing the Product Architecture

SRS is the reference for product architects to come out with the best architecture for the product to be developed.

Based on the requirements specified in SRS, usually more than one design approach for the product architecture is proposed and documented in a DDS - Design Document Specification.

Stage 4: Building or Developing the Product

In this stage of SDLC the actual development starts and the product is built. The programming code is generated as per DDS during this stage.

If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.

Developers must follow the coding guidelines defined by their organization and programming tools like compilers, interpreters, debuggers, etc. are used to generate the code.

Stage 5: Testing the Product

This stage is usually a subset of all the stages as in the modern SDLC models, the testing activities are mostly involved in all the stages of SDLC.

However, this stage refers to the testing only stage of the product where product defects are reported, tracked, fixed and retested, until the product reaches the quality standards

defined in the SRS.

Stage 6: Deployment in the Market and Maintenance

Once the product is tested and ready to be deployed it is released formally in the appropriate market. Sometimes product deployment happens in stages as per the business strategy of that organization.

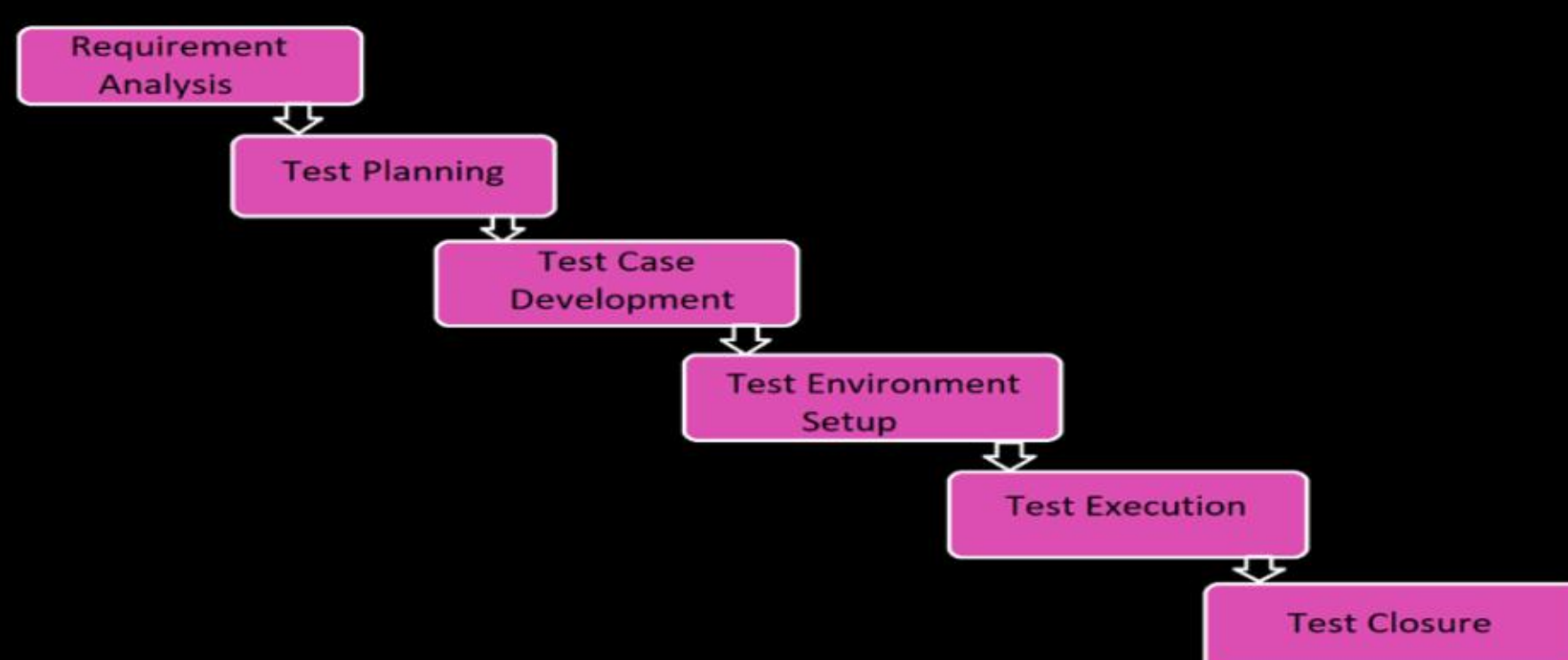
The product may first be released in a limited segment and tested in the real business environment (UAT- User acceptance testing).

Then based on the feedback, the product may be released as it is or with suggested enhancements in the targeting market segment.

After the product is released in the market, its maintenance is done for the existing customer base.

What is STLC?

- The Software Testing Life Cycle (STLC) is a systematic approach to testing a software application to ensure that it meets the requirements and is free of defects.
- It is a process that follows a series of steps or phases, and each phase has specific objectives and deliverables. The STLC is used to ensure that the software is of high quality, reliable, and meets the needs of the end-users.
- The main goal of the STLC is to identify and document any defects or issues in the software application as early as possible in the development process.
- This allows for issues to be addressed and resolved before the software is released to the public.



Requirement Analysis

- It is the first phase of STLC which deals with the QA

team analysing and understanding the requirement document to start the testing process with accurate information.

- In the Requirement analysis phase, the entry criteria are requirement documents, acceptance criteria, and understanding of the product architecture.
- The exit criteria for this phase are approved Requirements Traceability Matrix (RTM) and automation feasibility reports that evaluate if the tests can be automated.

Test Planning

- After determining the user requirements, the next phase of STLC is to plan the tests. It is a critical step of STLC where all the test plans are defined, and a detailed strategy is laid out for the project.
- It includes developing test steps, assigning roles and responsibilities, and choosing the right testing approach and automation tools, if any.
- The QA team also calculates the estimated cost and resources to begin the testing.

Test Case Development/Design

- After finishing the planning stage, this phase is designing and developing the test cases.
- In this step, the testing team creates the test steps with proper input, test execution conditions, and expected output for execution.
- For the test case design and development phase, the entry criteria are approved test plans, automation scripts, and defining the cost analysis of the complete project.

Test Environment Setup

- This phase includes setting up a test environment to run your test cases and check the application for its efficiency.
- It decides the conditions to test your application and involves configuring and deploying the test environment. Often, this step also includes setting up test servers.

- You can either perform the test environment setup in sequence or execute it after the requirement analysis phase, as it is independent of all the previous phases.

Test Execution

- After all the setup and planning are over, the test execution phase comes. The QA team runs the test cases and scripts to check the quality of the application.
- They compare the expected results with the actual results to understand which areas have issues and need attention.
- The entry criteria of this step are the combined exit criteria of all the previous steps, including test plans/cases, test environment, user requirement document, and more.

Test Closure

- The last phase of STLC is test closure which pertains to preparing a test result report. It summarizes the testing process and highlights the comparison between the actual and expected test results.
- The comparison includes the fulfilled project objectives, time taken to complete the testing, budget, test coverage, and identified issues.
- The entry criteria for the test closure phase are test results after executing all the test cases. The exit criteria are designed, delivered, and approved test results report.

Functional Testing:

- It is a type of software testing which is used to verify the functionality of the software application, whether the function is working according to the requirement specification.
- In functional testing, each function tested by giving the value, determining the output, and verifying the actual output with the expected value.
- Functional testing performed as black-box testing which is presented to confirm that the functionality of an application or system behaves as we are expecting. It is

done to verify the functionality of the application.

- Functional testing also called as black-box testing, because it focuses on application specification rather than actual code. Tester has to test only the program rather than the system.
- The main objective of functional testing is to test the functionality of the component.
- Functional testing is divided into multiple parts.
- Here are the following types of functional testing

1. Unit Testing:

- Unit testing is a type of software testing, where the individual unit or component of the software tested Unit testing, examine the different part of the application, by unit testing functional testing also done, because unit testing ensures each module is working correctly.
- The developer does unit testing. Unit testing is done in the development phase of the application.

2. Sanity Testing:

- Sanity testing involves the entire high-level business scenario is working correctly. Sanity testing is done to check the functionality/bugs fixed.
- Sanity testing is little advance than smoke testing.
- For example, login is working fine; all the buttons are working correctly; after clicking on the button navigation of the page is done or not.

3. Regression Testing:

- This type of testing concentrate to make sure that the code changes should not side effect the existing functionality of the system.
- Regression testing specifies when bug arises in the system after fixing the bug, regression testing concentrate on that all parts are working or not.
- Regression testing focuses on is there any impact on the system.

4. Integration Testing:

- Integration testing combined individual units and tested as a group. The purpose of this testing is to expose the faults in the interaction between the integrated units.
- Developers and testers perform integration testing.

What are the advantages of Functional Testing?

Advantages of functional testing are:

- It produces a defect-free product.
- It ensures that the customer is satisfied.
- It ensures that all requirements met.
- It ensures the proper working of all the functionality of an application/software/product.
- It ensures that the software/ product work as expected.
- It ensures security and safety.

What are the disadvantages of functional testing?

Disadvantages of functional testing are:

- Functional testing can miss a critical and logical error in the system.
- This testing is not a guarantee of the software to go live.
- The possibility of conducting redundant testing is high in functional testing.

What are the Boundary Values?

- Boundary value analysis is one of the widely used case design technique for black box testing. It is used to test boundary values because the input values near the boundary have higher chances of error.
- Whenever we do the testing by boundary value analysis, the tester focuses on, while entering boundary value whether the software is producing correct output or not.
- Boundary values are those that contain the upper and lower limit of a variable. Assume that, age is a variable

of any function, and its minimum value is 18 and the maximum value is 30, both 18 and 30 will be considered as boundary values.

- The boundary values include the values present on the boundary, the values just above the boundary (lower range), and values just below the boundary (upper range).
- Let us take an example of a variable x having values from 1 to 10.
- So the values of x are in the range of $1 \leq x \leq 10$. Its boundary values which should be tested are 1, 10, 2, and 9. Moreover, it is a good idea to include one intermediate value in a boundary. In this example, we can include the value 5.
- The final values which are considered for valid test cases are 1, 2, 5, 9, and 10. The values which are considered for invalid test cases can be 0 and 11.

For each variable, the below values should be considered for testing –

- Minimum value or lower value.
- Just above the minimum value.
- Nominal value or middle value.
- Just below maximum value.
- Maximum value or upper boundary.

What is Equivalence Class Testing?

- Equivalence Class Testing is an important software testing technique, used by the testers for grouping and partitioning the test input data, that is further used for the purpose of testing the software product, into a number of different classes.
- Moreover, equivalence class testing can be termed as

a logical step in the model of the functional testing.

- It Improves the quality of test cases, which further enhances the quality of testing, by removing the vast amount of redundancy and gaps that appear in the boundary value testing

Important Features of testing

- It is a Black box testing technique
- Restricts the testers to examine the software product, externally.
- Used to form groups of test inputs of similar behaviour or nature. Used at any level of STLC, i.e Unit, Integration.
- Test Cases are based on classes, which reduces the time and efforts required to
- Build a large large number of test cases.
- Equivalence class testing can be termed as a logical step in the model of functional testing. It improves the quality of test cases, which further enhances the quality of testing, by removing the vast amount of redundancy and gaps that appear in the boundary value testing. Other features of this testing technique are:
- It is a black box testing technique which restricts the testers to examine the software product, externally.
- Also known by the name of equivalence class partitioning, it is used to form groups of test inputs of similar behaviour or nature.

Advantages:

- Equivalence class testing helps reduce the number of test cases, without compromising the test coverage.
- Reduces the overall test execution time as it minimizes the set of test data.
- It can be applied to all levels of testing, such as unit testing, integration testing, system testing, etc.

- Enables the testers to focus on smaller data sets, which increases the probability to uncovering more defects in the software product.

Disadvantages:

- It does not consider the conditions for boundary value.
- The identification of equivalence classes relies heavily on the expertise of testers.

Decision Table testing:

- Decision tables are used in various engineering fields to represent complex logical relationships.
- This testing is a very effective tool in testing the software and its requirements management.
- The output may be dependent on many input conditions and decision tables give a tabular view of various combinations of input conditions and these conditions are in the form of True(T) and False(F).
- Also, it provides a set of conditions and its corresponding actions required in the testing.

Parts of Decision Tables :

The decision table has 4 parts which are divided into portions and are given below :

	Stubs	Entries
Condition	c1 c2 c3	
Action	a1 a2 a3 a4	

- **Condition Stubs** : The conditions are listed in this first upper left part of the decision table that is used to determine a particular action or set of actions.
- **Action Stubs** : All the possible actions are given in the first lower left portion (i.e, below condition stub) of the decision table.
- **Condition Entries** : In the condition entry, the values are inputted in the upper right portion of the decision table. In the condition entries part of the table, there are multiple rows and columns which are known as Rule.

- **Action Entries** : In the action entry, every entry has some associated action or set of actions in the lower right portion of the decision table and these values are called outputs.

Types of Decision Tables :

The decision tables are categorized into two types and these are given below:

- **Limited Entry** : In the limited entry decision tables, the condition entries are restricted to binary values.
- **Extended Entry** : In the extended entry decision table, the condition entries have more than two values. The decision tables use multiple conditions where a condition may have many possibilities instead of only ‘true’ and ‘false’ are known as extended entry decision tables.

Example : Library

User Registered	F	F	F	F	T	T	T	T
No Outstanding Fess	F	F	T	T	F	F	T	T
Under Borrow Limit?	F	T	F	T	F	T	F	T
Borrow Book?	NO	NO	NO	NO	NO	NO	NO	YES

Advantages of Decision Table Testing:

1. **Comprehensive Coverage:** Helps ensure all combinations of inputs are covered, reducing the risk of missing important test cases.
2. **Structured Testing:** Clearly identifies conditions and expected results, making the test case design more structured.
3. **Simplifies Complex Logic:** Helps simplify the testing of complex systems with multiple rules and conditions.

Retrospective on Functional Testing

A functional testing retrospective often involves discussions around the following topics:

1. Test Coverage

What worked well:

Comprehensive test coverage helped detect critical defects early in the process.

Clear mapping of test cases to requirements ensured that all functionality was tested.

Challenges:

Difficulty in maintaining coverage due to rapidly changing requirements.

Missing edge cases or overlooked areas due to incomplete requirement documentation.

Future Improvements:

Better requirement traceability matrix (RTM) to ensure every feature is covered by test cases.

More focus on exploratory testing for unplanned behaviors.

2. Test Design and Execution

What worked well:

Test scenarios were designed based on user stories and business requirements, ensuring user-centered tests.

Automation of repetitive test cases reduced manual effort and allowed regression tests to be run more frequently.

Challenges:

Test case design often lagged behind development due to insufficient collaboration between testers and developers.

Manual testing consumed significant resources, especially for complex workflows.

Future Improvements:

Early collaboration between testers and developers in the design phase (shift-left testing).

Greater focus on test automation, especially for regression tests and smoke tests.

3. Defect Detection and Management

What worked well:

Functional testing identified high-priority defects early in the release cycle, improving the overall stability of the software.

Use of tools for defect tracking and test management (e.g., JIRA, TestRail) streamlined bug reporting.

Challenges:

Some critical defects were only caught during late-stage

testing or after production release.

Inconsistent defect triaging led to delays in fixing high-priority bugs.

Future Improvements:

Introduce continuous testing across all stages of development to detect defects earlier (e.g., integrating testing into CI/CD pipelines).

Improve the defect triaging process by assigning more dedicated resources for defect prioritization.

4. Tools and Technology

What worked well:

Functional testing tools (e.g., Selenium, Cucumber, TestNG) enabled efficient test automation and faster test execution.

Cloud-based testing environments made it easier to execute tests across multiple platforms and browsers.

Challenges:

Tool integration issues with legacy systems, causing delays in test execution.

Maintenance of test scripts became cumbersome, especially when the application UI or backend changed frequently.

Future Improvements:

Use more robust and flexible automation tools that are compatible with evolving technologies (e.g., AI-driven testing tools like Testim or Mabl).

Implement better version control for test scripts to keep them up to date with application changes.

5. Collaboration and Communication

What worked well:

Frequent feedback loops between QA, development, and product teams helped address bugs and functional issues quickly.

Daily standups and sprint reviews enabled clear visibility into testing progress and blockers.

Challenges:

Communication gaps between teams, especially in distributed teams, sometimes led to misaligned expectations.

Testing often started too late in the development process, reducing the time available to fix and retest issues.

Future Improvements:

Encourage earlier and more frequent tester involvement in the development process.

Improve collaboration using agile practices such as pair testing, cross-functional teams, and continuous communication.

6. Time and Resource Management

What worked well:

Test automation saved significant time for regression testing, allowing teams to focus on more complex manual tests.

Parallel testing across environments reduced overall testing time.

Challenges:

Resource constraints led to bottlenecks during peak testing periods, especially towards the end of the sprint.

Delays in test data setup slowed down the testing process.

Future Improvements:

Better resource planning to ensure sufficient capacity during critical testing phases.

Automate test data generation and environment setup to streamline the testing process.

7. Impact on Product Quality

What worked well:

Functional testing improved the quality of releases by ensuring that critical business requirements were met.

End-user satisfaction was achieved as key workflows were thoroughly tested.

Challenges:

Functional testing sometimes missed non-functional aspects (e.g., performance, security) which led to post-release issues.

Future Improvements:

Combine functional testing with non-functional testing early in the development cycle (performance, security, etc.).

Perform risk-based testing to prioritize high-risk areas and prevent critical defects.